

LUCRAREA NR. 10

SUB-PROGRAME

1. Scopul lucrării

Lucrarea urmărește însușirea noțiunilor de bază referitoare la sub-programe în VHDL. Există două tipuri de sub-programe: *procedurile* și *funcțiile*, fiecare dintre ele fiind analizate și comentate în detaliu. Se discută de asemenea problema *supraîncărcării* atât a funcțiilor și a procedurilor, cât și a operatorilor.

2. Considerații teoretice

2.1 Descrierea concurentă

Sub-programele permit scrierea unor algoritmi reutilizabili. Valorile parametrilor pot fi diferite la fiecare apel, obținându-se astfel efecte diferite. Sub-programele pot fi folosite pentru mărirea lizibilității unui program, modularizând codul sursă, chiar dacă unele dintre ele nu vor fi apelate decât o singură dată.

În practică, sub-programele se folosesc atunci când apare necesitatea unei secvențe de instrucțiuni (așadar, respectivele instrucțiuni vor fi secvențiale și nu concurente) pentru descrierea unei anumite operații complexe, a unei conversii, a anumitor porțiuni din descrierea unor procese sau a unor funcții de rezoluție pentru semnale multi-sursă (funcțiile de rezoluție au fost prezentate pe larg în lucrarea nr. 8).

Există două tipuri de sub-programe:

- *procedurile* (cuvântul cheie **procedure**);
- *funcțiile* (cuvântul cheie **function**).

Procedurile pot acționa prin *efecte laterale*. Efectul lateral este, prin definiție, o modificare (sau o consultare) a mediului printr-un mijloc care diferă de parametrii de ieșire ai procedurilor, semnalele de mod **out** ale entităților sau valorile returnate de către funcții. De exemplu, asignarea unei valori unui semnal global (care nu este declarat local, ci în cadrul unui

pachet) constituie un efect lateral. Procedurile pot modifica, eventual, și valoarea parametrilor transmiși la apel.

Funcțiile returnează un rezultat (și numai unul) și nu acționează prin efecte laterale. Ele nu pot citi sau scrie variabile sau semnale decât dacă acestea sunt declarate în zona de declarații a funcției sau în zona declarativă a sub-programelor apelate. Cu alte cuvinte, dacă funcția va fi apelată de mai multe ori cu aceleași valori ale parametrilor, ea va returna de fiecare dată același rezultat.

Observație

O excepție la această regulă o constituie funcția NOW care returnează data curentă a simulării și care nu are parametri. Această funcție se numește *impură*.

Începând cu VHDL'93 a fost introdusă o nouă categorie de funcții, numite *impure* care au posibilitatea de a crea efecte laterale. Ele se declară folosind cuvântul cheie **impure**, fiind totodată supuse unor restricții asupra utilizării lor:

- o funcție de rezoluție nu poate fi o funcție impură;
- o funcție impură nu poate fi apelată dintr-o funcție pură (cu alte cuvinte, ne-impură).

Compilerul va verifica întotdeauna aceste restricții. În mod simetric, s-a introdus și cuvântul cheie **pure**: el permite explicitarea caracterului pur al unei funcții. Acest caracter este totodată atribuit în mod implicit oricărei funcții, dacă nu se specifică cuvântul cheie **impure**.

Apelul unei proceduri este o instrucțiune, în vreme ce apelul unei funcții se trece, la fel ca o valoare, în membrul drept al simbolului de asignare.

Un sub-program este alcătuit din două părți: *declarația* (opțională) și *corpul* său.

Declarația unui sub-program (numită și „specificatie de sub-program”) va indica:

- *genul* sub-programului (procedură sau funcție);
- *numele* acestuia;
- *lista parametrilor* săi (parametrii formali). Pentru fiecare dintre acești parametri, declarația sub-programului va preciza *modul* (de intrare, de ieșire, de intrare / ieșire) și *tipul* său;

– *tipul valorii returnate* (în cazul unei funcții).

Corpul sub-programului va conține algoritmul implementat; el va fi perfect coerent cu declarația sub-programului. Compilatorul are sarcina de a garanta această compatibilitate.

2.2 Declarația de sub-program

Declarația de sub-program este opțională. De regulă, în partea de specificare a pachetelor, vom găsi declarația tuturor sub-programelor pe care respectivul pachet le exportă. Corpul acestor sub-programe se va găsi în corpul pachetului.

Declarația furnizează numeroase informații despre funcționalitatea sub-programului, în special dacă beneficiază de câteva linii de comentarii. Ea poate fi deci văzută ca „manualul de utilizare” a sub-programului.

Sintaxa declarațiilor de sub-program este următoarea:

```
-- Declarația unei proceduri
procedure nume_procedură (lista_parametrilor_formali);

-- Declarația unei funcții
{pure/impure} function nume_funcție (lista_parametri_formali)
return tipul_rezultatului;
```

În cazul procedurii, numele este pur și simplu un identificator (la fel ca în cazul variabilelor sau semnalelor).

În cazul funcției, numele poate fi un identificator, însă și simbolul unui operator (cum ar fi +, =, & sau **and**). În acest caz, simbolul se va scrie între ghilimele ("") și funcția va putea fi activată sub forma unui operator.

Lista parametrilor formali are următoarea sintaxă:

```
{clasă_obiect} nume_param_1{, nume_param_2}: {mod_transmitere}
type valoare_implicită;
```

această declarație putându-se repeta de un anumit număr de ori.

Există trei moduri de transmitere a parametrilor:

1. *Modul intrare* (cuvântul cheie **in**) este modul de transmitere implicit. El corespunde unor parametri care pot fi citați în cadrul sub-programului, dar care nu pot fi modificați. Prin urmare, în

interiorul acestui sub-program, ei nu pot apărea în membrul stâng al nici unei instrucțiuni de asignare și nici ca parametri de apel ai vreunei proceduri în alt mod decât modul **in**;

2. *Modul ieșire* (cuvântul cheie **out**) este modul unui parametru formal a cărui valoare este returnată de către sub-program. În cazul de față, este vorba despre o procedură, căci acest mod le este interzis parametrilor formali ai unei funcții (o funcție nu returnează decât valoarea sa). În interiorul unei proceduri, este interzisă citirea unui parametru formal având modul ieșire – prin urmare, acest parametru nu va fi niciodată întâlnit în membrul drept al unei instrucțiuni de asignare. În plus, el nu poate fi folosit nici ca parametru actual de intrare sau de intrare / ieșire în cazul apelării unui alt sub-program intern;
3. *Modul combinat intrare / ieșire* (cuvântul cheie **inout**) autorizează orice citire și scriere în interiorul unei proceduri. Din aceleași motive ca și cele expuse în cazul modului de ieșire (**out**), el nu poate fi un parametru formal al unei funcții.

Un parametru care posedă o valoare implicită poate fi omis la apelul unui sub-program. Orice semnal are întotdeauna o valoare implicită asociată. Valoarea implicită este acea valoare care îi este atribuită unui parametru atunci când apelul nu precizează nimic în această direcție. Această valoare este o expresie care trebuie să fie de același tip ca și parametrul și este evaluată o dată pentru totdeauna în decursul elaborării (înaintea execuției programului VHDL).

În VHDL există trei clase de obiecte:

- Constante;
- Variabile;
- Semnale.

Restricțiile existente asupra clasei unui parametru în funcție de modul său sunt date în tabelul de mai jos (figura 10.1):

Mod \ Clasă	Obiect de clasă constantă	Obiect de clasă variabilă	Obiect de clasă semnal
Mod			

Parametru de mod in în proceduri	Permis și luat implicit	Permis	Permis
Parametru de mod out în proceduri	Permis	Permis și luat implicit	Permis
Parametru de mod inout în proceduri	Permis	Permis și luat implicit	Permis
Parametru de mod in în funcții	Permis și luat implicit	NU	Permis
Parametru de mod out în funcții	NU	NU	NU
Parametru de mod inout în funcții	NU	NU	NU

Figura 10.1 Relația dintre clase și modul de transmitere a parametrilor formali

Sintaxa prezentată în figura 10.1 arată că, în cazul definirii unui parametru formal, nu este obligatoriu să se precizeze dacă este vorba despre o constantă, o variabilă sau un semnal. Tabelul de mai sus ilustrează clasele care sunt luate implicit în funcție de modul parametrului.

Iată câteva exemple de declarații de sub-programe:

```

procedure VERIF_PERIOADA (signal TACT: in BIT);
procedure CONVERSIE (INTRARE : in REAL; IEȘIRE: out INTEGER);
function CONVERSIE (INTRARE : in REAL) return INTEGER;
function "+" (A, B: INTEGER) return INTEGER;

```

Următoarele porțiuni de cod sunt variante perfect echivalente:

```

procedure MIN (A, B: INTEGER; C: out INTEGER);
procedure MIN (A: INTEGER; B: INTEGER; C: out INTEGER);
procedure MIN (A, B: in INTEGER; C: out INTEGER);

function MIN (A, B: INTEGER) return INTEGER;
function MIN (A: INTEGER; B: INTEGER) return INTEGER;
function MIN (A: in INTEGER, B: in INTEGER) return INTEGER;

```

2.3 Corpul de sub-program

Corpul sub-programului are menirea de a realiza funcționalitatea sub-programului. Sintaxa sa este următoarea:

```
Antet_sub-program is  
{zona declarativă} --Acoladele delimitează părțile opționale  
begin  
{zona rezervată instrucțiunilor}  
end {nume_sub-program}
```

Corpul unui sub-program va conține algoritmul atașat sub-programului respectiv. Antetul acestui corp începe prin copierea specificației sub-programului, singura diferență constând în faptul că ea se încheie cu cuvântul cheie **is** și nu cu punct și virgulă („;”).

Urmează apoi o parte declarativă care va conține declarațiile interne acestui sub-program. Declarațiile nu vor fi vizibile din exteriorul sub-programului.

În această zonă se pot găsi și alte declarații sau corpuri de sub-program, tipuri și sub-tipuri, constante și variabile, fișiere, alias-uri și attribute. Este permisă specificarea de attribute și prezența clauzei **use**. Cea mai importantă restricție o constituie aceea referitoare la semnale: în cadrul unui sub-program este interzisă declararea semnalelor, deoarece ne aflăm în domeniul secvențial (aici, după încheierea execuției sale, sub-programul „moare”, în timp ce semnalul ar continua să existe!). Zona de declarații se termină o dată cu cuvântul cheie **begin**, care indică începutul porțiunii care conține algoritmul sub-programului.

Această porțiune este alcătuită dintr-o suită de instrucțiuni care se execută secvențial. Ea începe deci cu cuvântul cheie **begin** și se încheie cu cuvântul cheie **end**, urmat eventual de numele sub-programului. În zona de algoritmi, o funcție returnează valoarea sa prin intermediul instrucțiunii **return**, pe când o procedură va asigura valori parametrilor de mod **out** și **inout**.

De exemplu: *funcția* MIN returnează cel mai mic dintre cei doi întregi pe care îi primește ca parametri. Partea sa declarativă (opțională) este vidă. Se folosește instrucțiunea de selecție simplă **if ... then... else... end if**.

```
function MIN (A, B: INTEGER) return INTEGER is  
begin
```

```
    if A < B then
        return A;
    else
        return B;
    end if;
end MIN;
```

Să examinăm în continuare *procedura* MIN. Partea declarativă a rămas în continuare vidă:

```
-- Prima variantă a procedurii MIN
procedure MIN (A, B: in INTEGER; C: out INTEGER) is
begin
    if A < B then
        C := A;
    else
        C := B;
    end if;
end MIN;
-- A doua variantă a procedurii MIN: se folosește în plus o
-- variabilă locală (numai din considerente demonstrative)
procedure MIN (A, B: in INTEGER; C: out INTEGER) is
variable VAR_LUCRU: INTEGER;
begin
    if A < B then
        VAR_LUCRU := A;
    else
        VAR_LUCRU := B;
    end if;
    C := VAR_LUCRU;
end MIN;
```

2.4 Apelul de sub-program

Deși un sub-program este de natură secvențială (instrucțiunile pe care le conține sunt executate secvențial), apelul său se poate face în două moduri:

- a) ca o acțiune secvențială;
- b) ca o acțiune concurentă.

Există două moduri de a indica parametrii unui sub-program în momentul apelării sale: *prin poziție* sau *prin denumire*.

Apelul prin poziție constă în a enumera toți parametrii, în ordine, separați prin virgule. Corespondența dintre parametrii actuali și parametrii formali se efectuează în funcție de această ordine.

Apelul prin denumire constă în a indica numele parametrului formal înaintea fiecărui parametru actual (parametrul de apel). Corespondența este atunci explicită și se utilizează simbolul „=>”. În acest caz, ordinea parametrilor nu mai este semnificativă. Apelul prin denumire se utilizează pentru ameliorarea lizibilității programelor sau pentru a nu modifica decât câțiva parametri, presupunându-se că ceilalți au o valoare implicită.

Prezentăm în continuare câteva exemple: procedura și respectiv funcția MIN va fi apelată în ambele moduri (prin poziție și prin denumire).

```
-- Procedura MIN
MIN (VAR, 5, REZULTAT); -- Apel pozițional
MIN (A => VAR, B => 5, C => REZULTAT); -- Apel prin denumire
MIN (B => 5, C => REZULTAT, A => VAR); -- Apel prin denumire

-- Funcția MIN
REZULTAT := MIN(VAR, 5); -- Apel pozițional
REZULTAT := MIN(A => VAR, B => 5); -- Apel prin denumire
```

2.5 Supraîncărcarea

Două sub-programe se numesc *supraîncărcate* dacă au același nume, însă profilurile lor diferă.

Profilul unui sub-program este un ansamblu de informații care cuprinde numărul, ordinea și tipul parametrilor formali, precum și – în cazul funcțiilor – tipul rezultatului returnat.

Supraîncărcarea a două sub-programe de profiluri diferite este deosebit de utilă și mărește ușurința scrierii de cod sursă VHDL, asigurând în același timp o foarte bună lizibilitate a programelor. Pentru a determina sub-programul pe care trebuie efectiv să-l apeleze, compilatorul va alege profilul care corespunde apelului. Dacă nici un profil nu corespunde sau dacă există mai multe profiluri identice candidate, acest fapt va fi semnalat ca eroare la compilare.

Observație

Anumite informații conținute în specificația unui sub-program, cum ar fi numele parametrilor formali, modul lor de transmitere, clasa lor sau valoarea lor implicită nu fac parte din profilul sub-programului. Două sub-programe care nu diferă decât prin una sau mai multe dintre aceste informații vor provoca o ambiguitate care va fi semnalată de către compilator. Prin urmare, următoarele două funcții nu se vor supraîncărca (se va genera o eroare la compilare, deoarece simbolul MIN este definit de două ori):

```
function MIN(A, B: INTEGER) return INTEGER;
function MIN(C, D: INTEGER) return INTEGER;
```

Iată câteva exemple de supraîncărcare:

```
-- Supraîncărcarea a trei proceduri
procedure CONVERSIE (INTRARE: in REAL, IEȘIRE: out INTEGER);
procedure CONVERSIE (INTRARE: in BIT, IEȘIRE: out INTEGER);
procedure CONVERSIE (INTRARE: in OCTET, IEȘIRE: out INTEGER);

-- Supraîncărcarea a trei funcții
function MIN(A, B: INTEGER) return INTEGER;
function MIN(A: REAL, B: REAL) return REAL;
function MIN(A: in BIT, B: in BIT) return BIT;
```

Supraîncărcarea poate fi aplicată și operatorilor. De exemplu, putem supraîncărca operatorul logic **and** folosind următoarea funcție:

```
function "AND" (A,B: TIP_LOGIC) return BOOLEAN;
```

Această declarație va autoriza scrierea unor expresii precum:

```
A and B
```

unde A și B sunt de tipul TIP_LOGIC.

Funcția astfel definită trebuie să aibă respectiv unul singur sau doi parametri, în funcție de tipul operatorului: unar sau binar. Operatorii „+” și „-” pot fi supraîncărcați atât ca operatori binari cât și ca operatori unari. Utilizarea acestei funcții va putea întotdeauna să se efectueze sub forma sa clasică:

```
"AND" (A, B) -- Nu trebuie uitate ghilimelele!
```

Supraîncărcarea operatorilor prezintă un interes deosebit pentru simplificarea scrierii programelor care operează pe semnale de alte tipuri decât tipul BIT.

Iată trei exemple de supraîncărcare a operatorului „+”:

```
function "+" (A,B: BIT) return BIT;  
function "+" (A,B: BIT_VECTOR) return BIT_VECTOR;  
function "+" (A,B: IMPEDANȚĂ) return IMPEDANȚĂ;
```

2.6 Considerații suplimentare

2.6.1 Recursivitatea

În VHDL există posibilitatea de a scrie programe recursive. Într-un limbaj de programare clasic, în general vorbind, recursivitatea reprezintă proprietatea unui sub-program de a se putea apela pe el însuși. În VHDL este permisă și recursivitatea încrucișată (sub-programul A apelează sub-programul B, care la rândul său apelează sub-programul A). În cazul aplicării recursivității, trebuie să ne asigurăm că am scris o instrucțiune de test pentru stoparea sa.

Astfel, este posibil să descriem un sumator complet pe N biți în funcție de el însuși, instanțiat pe N-1 biți, și a unui modul de un singur bit care va avea rolul de test de stopare a recursivității.

Trebuie însă să avem în vedere că, în VHDL, scopul final îl reprezintă în general obținerea unei structuri hardware. De aceea, este bine să ne punem mereu întrebarea: la ce ne ajută acest gen de descriere? Va exista oare o realitate materială (hardware) corespunzătoare? În acest caz, răspunsul este din păcate negativ...

2.6.1 Vizibilitatea

În figura 10.2 sunt prezentate regulile de vizibilitate a sub-programelor în VHDL:

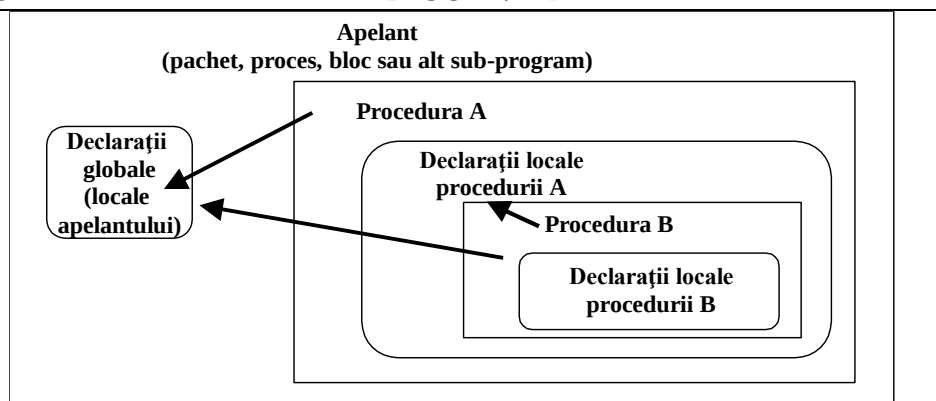


Figura 10.2 Reguli de vizibilitate

Declarațiile efectuate în interiorul unui sub-program nu sunt vizibile (deci nu sunt accesibile) din exteriorul acestuia, însă sunt vizibile de către procedurile înglobate. Săgețile din figura 10.2 pornesc de la obiectul care „vede” către obiectul „văzut”.

2.6.3 Logica sub-programelor

O procedură se execută până la **end**-ul său final.

O funcție nu trebuie să ajungă niciodată la instrucțiunea sa **end** finală, lucru care ar genera o eroare în cursul execuției. Ea trebuie să întâlnească întotdeauna cuvântul cheie **return** și să predea atunci controlul (împreună cu valoarea pe care trebuie s-o returneze) apelantului.

3. Desfășurarea lucrării

- 3.1 Se vor supraîncărca operatorii de adunare și scădere definiți pe întregi pentru valori de tip `STD_LOGIC` și `STD_LOGIC_VECTOR`.
- 3.2 Se vor supraîncărca funcțiile `MIN` și `MAX` din lucrare pentru parametri de tip `STD_LOGIC` și `STD_LOGIC_VECTOR`.
- 3.3 Se vor defini funcțiile de conversie între următoarele tipuri de date: `BIT_VECTOR` și `INTEGER`; `BIT_VECTOR` și `STRING`.
- 3.4 Se va defini un sistem numeric care rezolvă ecuații de gradul 2 folosind proceduri.